```
/*
============================================================
 FARM TRACE - ESP8266 #1
 SENSOR + MQTT + DFR0602 OFFLINE STORAGE
============================================================

 ESP8266 #1 CONNECTIONS
 ----------------------

 DHT22:
  DATA -> D5

 BMP280 + MPU6050:
  SDA -> D2
  SCL -> D1

 GPS NEO-6M:
  GPS TX -> D6
  GPS RX -> NOT CONNECTED

 DFR0602:
  DFR TX -> D7
  DFR RX -> D8

 TAMPER:
  D3 -> SWITCH -> GND

============================================================
```

```
 DFR0602 PROTOCOL
============================================================

 PING
 STATUS
 WRITE,<record>
 READ
 ACK
 NACK
 CLEAR

 DFR0602 BAUD = 9600

============================================================
*/

#include <Arduino.h>
#include <ESP8266WiFi.h>
#include <PubSubClient.h>

#include <Wire.h>
#include <Adafruit_BMP280.h>
#include <Adafruit_MPU6050.h>
#include <Adafruit_Sensor.h>

#include <DHT.h>
#include <SoftwareSerial.h>
```

```
#include <EEPROM.h>

// ============================================================
// PIN DEFINITIONS
// ============================================================

#define DHT_PIN D5
#define DHT_TYPE DHT22

#define I2C_SDA D2
#define I2C_SCL D1

#define GPS_RX D6
#define GPS_TX_UNUSED -1

#define DFR_RX D7
#define DFR_TX D8

#define TAMPER_PIN D3

// ============================================================
// SENSOR OBJECTS
// ============================================================

DHT dht(DHT_PIN, DHT_TYPE);

Adafruit_BMP280 bmp;
Adafruit_MPU6050 mpu;
```

```
SoftwareSerial gpsSerial(GPS_RX, GPS_TX_UNUSED);
SoftwareSerial dfrSerial(DFR_RX, DFR_TX);

// ============================================================
// WIFI AP FOR ESP8266 #2
// ============================================================

const char* AP_SSID = "FARM_TRACE_NODE";
const char* AP_PASSWORD = "12345678";

WiFiServer displayServer(80);

// ============================================================
// INTERNET WIFI
// ============================================================

const char* WIFI_SSID = "iQOO";
const char* WIFI_PASSWORD = "vimal8877";

// ============================================================
// MQTT
// ============================================================

const char* MQTT_BROKER = "10.104.206.131";
const uint16_t MQTT_PORT = 1883;

const char* MQTT_TOPIC = "trace/node01";
```

```
const char* MQTT_ACK_TOPIC = "trace/node01/ack";

WiFiClient mqttWifiClient;
PubSubClient mqttClient(mqttWifiClient);

// ============================================================
// NODE
// ============================================================

const char* NODE_ID = "NODE01";

// ============================================================
// EEPROM
// ============================================================

#define EEPROM_SIZE 16

#define EEPROM_MAGIC_ADDR 0
#define EEPROM_ID_ADDR 4

#define EEPROM_MAGIC 0x53494835UL

// Latest existing record = 3334
// Therefore next record = 3335
const uint32_t DEFAULT_NEXT_RECORD_ID = 3335;

uint32_t nextRecordID = DEFAULT_NEXT_RECORD_ID;
```

```
// ============================================================
// SENSOR VARIABLES
// ============================================================

float dhtTemp = -999.0;
float dhtHum = -999.0;

float pressure = -999.0;
float bmpTemp = -999.0;

float accX = 0;
float accY = 0;
float accZ = 0;

float gyroX = 0;
float gyroY = 0;
float gyroZ = 0;

double gpsLat = 0.0;
double gpsLon = 0.0;

int gpsSat = 0;

bool gpsFix = false;

bool tamperState = false;

// ============================================================
```

```
// GPS
// ============================================================

String gpsLine = "";

String lastGPSDate = "";
String lastGPSTime = "";

// ============================================================
// MQTT ACK
// ============================================================

volatile bool mqttApplicationAck = false;
volatile uint32_t mqttAckRecordID = 0;

// ============================================================
// TIMERS
// ============================================================

unsigned long lastSensorRead = 0;

const unsigned long SENSOR_INTERVAL = 5000;

// ============================================================
// FORWARD DECLARATIONS
// ============================================================

void loadNextRecordID();
```

```
void saveNextRecordID();

void readSensors();

void readDHT();
void readBMP280();
void readMPU6050();
void readTamper();

void processGPS();
void parseGPSLine(String line);

bool connectMQTT();

void mqttCallback(
  char* topic,
  byte* payload,
  unsigned int length
);

bool publishOnlineRecord(
  String record,
  uint32_t recordID
);

bool writeDFRRecord(String record);

bool startOfflineSync();
```

```
bool readDFRQueue();

uint32_t extractRecordID(String record);

String createRecord(uint32_t recordID);

void handleDisplayServer();

// ============================================================
// SETUP
// ============================================================

void setup()
{
  Serial.begin(115200);

  delay(500);

  Serial.println();
  Serial.println();
  Serial.println("========================================");
  Serial.println(" FARM TRACE - ESP8266 #1");
  Serial.println(" SENSOR / MQTT / OFFLINE CONTROLLER");
  Serial.println("========================================");

  // ============================================================
  // TAMPER
  // ============================================================
```

```
pinMode(TAMPER_PIN, INPUT_PULLUP);

// ============================================================
// I2C
// ============================================================

Wire.begin(I2C_SDA, I2C_SCL);

Serial.println("I2C INITIALIZED");

// ============================================================
// DHT22
// ============================================================

dht.begin();

Serial.println("DHT22 INITIALIZED");

// ============================================================
// BMP280
// ============================================================

if (bmp.begin(0x76))
{
  Serial.println("BMP280 FOUND AT 0x76");
}
else if (bmp.begin(0x77))
```

```
{
 Serial.println("BMP280 FOUND AT 0x77");
}
else
{
 Serial.println("BMP280 NOT FOUND");
}

// ========================================================
// MPU6050
// ========================================================

if (mpu.begin())
{
 Serial.println("MPU6050 FOUND");

 mpu.setAccelerometerRange(
  MPU6050_RANGE_8_G
 );

 mpu.setGyroRange(
  MPU6050_RANGE_500_DEG
 );

 mpu.setFilterBandwidth(
  MPU6050_BAND_21_HZ
 );
}
```

```
else
{
  Serial.println("MPU6050 NOT FOUND");
}

// ============================================================
// GPS
// ============================================================

gpsSerial.begin(9600);

Serial.println("GPS SERIAL INITIALIZED @ 9600");

// ============================================================
// DFR0602
// ============================================================

dfrSerial.begin(9600);

Serial.println("DFR0602 SERIAL INITIALIZED @ 9600");

// ============================================================
// EEPROM
// ============================================================

EEPROM.begin(EEPROM_SIZE);

loadNextRecordID();
```

```
Serial.print("NEXT RECORD ID = ");
Serial.println(nextRecordID);

// ============================================================
// WIFI MODE
// ============================================================

WiFi.mode(WIFI_AP_STA);

// ============================================================
// ESP8266 #1 AP
// ============================================================

WiFi.softAP(
  AP_SSID,
  AP_PASSWORD
);

Serial.println();
Serial.println("ESP8266 #1 AP STARTED");

Serial.print("AP SSID: ");
Serial.println(AP_SSID);

Serial.print("AP IP: ");
Serial.println(
  WiFi.softAPIP()
```

```
);

// ========================================================
// DISPLAY SERVER
// ========================================================

displayServer.begin();

Serial.println("DISPLAY SERVER STARTED");

// ========================================================
// INTERNET WIFI
// ========================================================

WiFi.begin(
  WIFI_SSID,
  WIFI_PASSWORD
);

Serial.println();
Serial.print("CONNECTING TO INTERNET WIFI");

unsigned long wifiStart = millis();

while (
  WiFi.status() != WL_CONNECTED &&
  millis() - wifiStart < 15000
)
```

```
{
  delay(500);
  Serial.print(".");
}

Serial.println();

if (WiFi.status() == WL_CONNECTED)
{
  Serial.println("INTERNET WIFI CONNECTED");

  Serial.print("LOCAL IP: ");
  Serial.println(
    WiFi.localIP()
  );
}
else
{
  Serial.println("INTERNET WIFI NOT AVAILABLE");
  Serial.println("OFFLINE MODE READY");
}

// ============================================================
// MQTT
// ============================================================

mqttClient.setServer(
  MQTT_BROKER,
```

```
  MQTT_PORT
);

mqttClient.setCallback(
  mqttCallback
);

if (WiFi.status() == WL_CONNECTED)
{
  connectMQTT();
}

// ============================================================
// DFR0602 PING
// ============================================================

delay(1000);

Serial.println();
Serial.println("TESTING DFR0602...");

dfrSerial.listen();

while (dfrSerial.available())
{
  dfrSerial.read();
}
```

```
dfrSerial.println("PING");

unsigned long pingStart = millis();

bool pingOK = false;

while (
  millis() - pingStart < 3000
)
{
  if (dfrSerial.available())
  {
    String response =
      dfrSerial.readStringUntil('\n');

    response.trim();

    Serial.print("DFR0602: ");
    Serial.println(response);

    if (response == "PONG")
    {
      pingOK = true;
      break;
    }
  }

  yield();
```

```
 }

 if (pingOK)
 {
  Serial.println("DFR0602 PING OK");
 }
 else
 {
  Serial.println("DFR0602 PING FAILED");
 }

 // ============================================================
 // INITIAL SENSOR READ
 // ============================================================

 readSensors();

 Serial.println();
 Serial.println("========================================");
 Serial.println("SYSTEM READY");
 Serial.println("========================================");
}

// ============================================================
// LOOP
// ============================================================

void loop()
```

```
{
 // ============================================================
 // GPS
 // ============================================================

 processGPS();

 // ============================================================
 // MQTT
 // ============================================================

 if (WiFi.status() == WL_CONNECTED)
 {
  if (!mqttClient.connected())
  {
    connectMQTT();
  }

  mqttClient.loop();
 }

 // ============================================================
 // DISPLAY
 // ============================================================

 handleDisplayServer();

 // ============================================================
```

```
// SENSOR + RECORD
// ========================================================

if (
  millis() - lastSensorRead >=
  SENSOR_INTERVAL
)
{
  lastSensorRead = millis();

  readSensors();

  uint32_t currentID =
    nextRecordID;

  String record =
    createRecord(currentID);

  Serial.println();
  Serial.println("========================================");
  Serial.print("NEW RECORD: ");
  Serial.println(currentID);
  Serial.println("========================================");

  Serial.println(record);

  bool delivered = false;
```

```
// ========================================================
// ONLINE
// ========================================================

if (
  WiFi.status() == WL_CONNECTED &&
  mqttClient.connected()
)
{
  Serial.println(
    "MQTT ONLINE - TRYING DELIVERY"
  );

  delivered =
    publishOnlineRecord(
      record,
      currentID
    );

  if (delivered)
  {
    Serial.println(
      "ONLINE DELIVERY SUCCESS"
    );
  }
  else
  {
    Serial.println(
```

```
    "ONLINE DELIVERY FAILED"
   );
  }
 }
 else
 {
  Serial.println(
   "MQTT OFFLINE"
  );
 }

 // ========================================================
 // OFFLINE STORAGE
 // ========================================================

 if (!delivered)
 {
  Serial.println(
   "OFFLINE QUEUE MODE"
  );

  bool stored =
   writeDFRRecord(record);

  if (stored)
  {
   Serial.print(
    "OFFLINE RECORD STORED: "
```

```
    );

    Serial.println(
      currentID
    );
  }
  else
  {
    Serial.print(
      "OFFLINE SD WRITE FAILED: "
    );

    Serial.println(
      currentID
    );
  }
}

// ========================================================
// NEXT ID
// ========================================================

nextRecordID++;

saveNextRecordID();

Serial.print(
  "NEXT RECORD ID = "
```

```
    );

    Serial.println(
      nextRecordID
    );

    // ========================================================
    // TRY OLD OFFLINE QUEUE
    // ========================================================

    if (
      WiFi.status() == WL_CONNECTED &&
      mqttClient.connected()
    )
    {
      startOfflineSync();
    }
  }

  yield();
}

// ============================================================
// EEPROM LOAD
// ============================================================

void loadNextRecordID()
{
```

```
uint32_t magic = 0;

EEPROM.get(
 EEPROM_MAGIC_ADDR,
 magic
);

if (magic != EEPROM_MAGIC)
{
 Serial.println(
  "EEPROM MAGIC NOT FOUND"
 );

 nextRecordID =
  DEFAULT_NEXT_RECORD_ID;

 EEPROM.put(
  EEPROM_MAGIC_ADDR,
  EEPROM_MAGIC
 );

 EEPROM.put(
  EEPROM_ID_ADDR,
  nextRecordID
 );

 EEPROM.commit();
```

```
  Serial.print(
    "EEPROM INITIALIZED: "
  );

  Serial.println(
    nextRecordID
  );

  return;
}

EEPROM.get(
  EEPROM_ID_ADDR,
  nextRecordID
);

// Never allow counter to go backwards.
if (
  nextRecordID <
  DEFAULT_NEXT_RECORD_ID
)
{
  nextRecordID =
    DEFAULT_NEXT_RECORD_ID;

  EEPROM.put(
    EEPROM_ID_ADDR,
    nextRecordID
```

```
    );

    EEPROM.commit();
  }

  Serial.print(
    "EEPROM NEXT ID: "
  );

  Serial.println(
    nextRecordID
  );
}

// ============================================================
// EEPROM SAVE
// ============================================================

void saveNextRecordID()
{
  EEPROM.put(
    EEPROM_ID_ADDR,
    nextRecordID
  );

  EEPROM.commit();
}
```

```
// ============================================================
// READ ALL SENSORS
// ============================================================

void readSensors()
{
  readDHT();

  readBMP280();

  readMPU6050();

  readTamper();

  processGPS();

  Serial.println();
  Serial.println("----------- SENSOR DATA -----------");

  Serial.print("TEMP = ");
  Serial.println(dhtTemp);

  Serial.print("HUM = ");
  Serial.println(dhtHum);

  Serial.print("PRESSURE = ");
  Serial.println(pressure);
```

```
Serial.print("BMP_TEMP = ");
Serial.println(bmpTemp);

Serial.print("ACC_X = ");
Serial.println(accX);

Serial.print("ACC_Y = ");
Serial.println(accY);

Serial.print("ACC_Z = ");
Serial.println(accZ);

Serial.print("GYRO_X = ");
Serial.println(gyroX);

Serial.print("GYRO_Y = ");
Serial.println(gyroY);

Serial.print("GYRO_Z = ");
Serial.println(gyroZ);

Serial.print("GPS_FIX = ");
Serial.println(
  gpsFix ? "1" : "0"
);

Serial.print("LAT = ");
Serial.println(
```

```
  gpsLat,
  6
 );

 Serial.print("LON = ");
 Serial.println(
  gpsLon,
  6
 );

 Serial.print("SAT = ");
 Serial.println(
  gpsSat
 );

 Serial.print("TAMPER = ");
 Serial.println(
  tamperState ? "1" : "0"
 );

 Serial.println("----------------------------------");
}

// ============================================================
// DHT
// ============================================================

void readDHT()
```

```
{
  float t =
    dht.readTemperature();

  float h =
    dht.readHumidity();

  if (
    isnan(t) ||
    isnan(h)
  )
  {
    dhtTemp = -999.0;
    dhtHum = -999.0;

    Serial.println(
      "DHT22 READ FAILED"
    );
  }
  else
  {
    dhtTemp = t;
    dhtHum = h;
  }
}

// ============================================================
// BMP280
```

```
// ============================================================

void readBMP280()
{
 float p =
  bmp.readPressure();

 float t =
  bmp.readTemperature();

 if (
  isnan(p) ||
  isnan(t)
 )
 {
  pressure = -999.0;
  bmpTemp = -999.0;

  Serial.println(
   "BMP280 READ FAILED"
  );
 }
 else
 {
  pressure =
   p / 100.0;

  bmpTemp =
```

```
    t;
  }
}

// ============================================================
// MPU6050
// ============================================================

void readMPU6050()
{
  sensors_event_t accel;
  sensors_event_t gyro;
  sensors_event_t temp;

  mpu.getEvent(
    &accel,
    &gyro,
    &temp
  );

  accX =
    accel.acceleration.x;

  accY =
    accel.acceleration.y;

  accZ =
    accel.acceleration.z;
```

```
  gyroX =
    gyro.gyro.x;

  gyroY =
    gyro.gyro.y;

  gyroZ =
    gyro.gyro.z;
}

// ============================================================
// TAMPER
// ============================================================

void readTamper()
{
  tamperState =
    (digitalRead(TAMPER_PIN) == LOW);
}

// ============================================================
// GPS PROCESS
// ============================================================

void processGPS()
{
  gpsSerial.listen();
```

```
while (
 gpsSerial.available()
)
{
 char c =
  gpsSerial.read();

 if (c == '\n')
 {
  gpsLine.trim();

  if (gpsLine.length() > 0)
  {
   parseGPSLine(
    gpsLine
   );
  }

  gpsLine = "";
 }
 else if (c != '\r')
 {
  gpsLine += c;

  if (gpsLine.length() > 180)
  {
   gpsLine = "";
```

```
    }
   }
  }
}

// ============================================================
// GPS GGA PARSER
// ============================================================

void parseGPSLine(String line)
{
 if (
  !line.startsWith("$GPGGA") &&
  !line.startsWith("$GNGGA")
 )
 {
  return;
 }

 String fields[15];

 int fieldIndex = 0;

 int start = 0;

 for (
  int i = 0;
  i <= line.length();
```

```
  i++
)
{
  if (
    i == line.length() ||
    line[i] == ','
  )
  {
    if (fieldIndex < 15)
    {
      fields[fieldIndex] =
        line.substring(
          start,
          i
        );
    }

    fieldIndex++;

    start =
      i + 1;
  }
}

if (fieldIndex < 8)
{
  return;
}
```

```
// ============================================================
// UTC
// ============================================================

if (
 fields[1].length() >= 6
)
{
 lastGPSTime =
  fields[1];
}

// ============================================================
// FIX
// ============================================================

int fixQuality =
 fields[6].toInt();

gpsFix =
 (fixQuality > 0);

// ============================================================
// SATELLITES
// ============================================================

gpsSat =
```

```
 fields[7].toInt();

// ========================================================
// LATITUDE
// ========================================================

if (
 fields[2].length() > 0 &&
 fields[3].length() > 0
)
{
 double rawLat =
  fields[2].toDouble();

 int degrees =
  (int)(
   rawLat / 100.0
  );

 double minutes =
  rawLat -
  (degrees * 100.0);

 gpsLat =
  degrees +
  minutes / 60.0;

 if (
```

```
   fields[3] == "S"
  )
  {
   gpsLat =
    -gpsLat;
  }
}

// ========================================================
// LONGITUDE
// ========================================================

if (
  fields[4].length() > 0 &&
  fields[5].length() > 0
)
{
  double rawLon =
   fields[4].toDouble();

  int degrees =
   (int)(
    rawLon / 100.0
   );

  double minutes =
   rawLon -
   (degrees * 100.0);
```

```
    gpsLon =
      degrees +
      minutes / 60.0;

    if (
      fields[5] == "W"
    )
    {
      gpsLon =
        -gpsLon;
    }
  }
}

// ============================================================
// MQTT CONNECTION
// ============================================================

bool connectMQTT()
{
  if (
    mqttClient.connected()
  )
  {
    return true;
  }
```

```
if (
 WiFi.status() != WL_CONNECTED
)
{
 return false;
}

Serial.println();
Serial.println(
 "CONNECTING MQTT..."
);

String clientID =
 String("FARMTRACE_") +
 String(
  ESP.getChipId(),
  HEX
 );

if (
 mqttClient.connect(
  clientID.c_str()
 )
)
{
 Serial.println(
  "MQTT CONNECTED"
 );
```

```
 bool subscribed =
  mqttClient.subscribe(
   MQTT_ACK_TOPIC
  );

 if (subscribed)
 {
  Serial.println(
   "MQTT ACK TOPIC SUBSCRIBED"
  );
 }
 else
 {
  Serial.println(
   "MQTT ACK SUBSCRIBE FAILED"
  );
 }

 return true;
}

Serial.print(
 "MQTT CONNECTION FAILED: "
);

Serial.println(
 mqttClient.state()
```

```
  );

  return false;
}

// ============================================================
// MQTT CALLBACK
// ============================================================

void mqttCallback(
  char* topic,
  byte* payload,
  unsigned int length
)
{
  String message = "";

  for (
    unsigned int i = 0;
    i < length;
    i++
  )
  {
    message +=
      (char)payload[i];
  }

  message.trim();
```

```
Serial.println();
Serial.println(
  "MQTT ACK RECEIVED"
);

Serial.print(
  "TOPIC: "
);

Serial.println(
  topic
);

Serial.print(
  "DATA: "
);

Serial.println(
  message
);

if (
  message.startsWith("ACK,")
)
{
  String idText =
    message.substring(4);
```

```
    idText.trim();

    uint32_t id =
      idText.toInt();

    mqttAckRecordID =
      id;

    mqttApplicationAck =
      true;

    Serial.print(
      "APPLICATION ACK: "
    );

    Serial.println(
      id
    );
  }
}

// ============================================================
// PUBLISH ONLINE RECORD
// ============================================================

bool publishOnlineRecord(
  String record,
```

```
  uint32_t recordID
)
{
  if (
    !mqttClient.connected()
  )
  {
    Serial.println(
      "MQTT NOT CONNECTED"
    );

    return false;
  }

  mqttApplicationAck =
    false;

  mqttAckRecordID =
    0;

  Serial.println();
  Serial.print(
    "PUBLISHING RECORD "
  );

  Serial.println(
    recordID
  );
```

```
bool published =
  mqttClient.publish(
    MQTT_TOPIC,
    record.c_str()
  );

if (!published)
{
  Serial.println(
    "MQTT PUBLISH FAILED"
  );

  return false;
}

Serial.println(
  "MQTT PUBLISH SUCCESS"
);

// ------------------------------------------------------
// Wait for backend ACK
// ------------------------------------------------------

unsigned long start =
  millis();

while (
```

```
  millis() - start < 5000
)
{
  mqttClient.loop();

  if (
    mqttApplicationAck &&
    mqttAckRecordID == recordID
  )
  {
    Serial.print(
      "BACKEND ACK RECEIVED: "
    );

    Serial.println(
      recordID
    );

    return true;
  }

  yield();
}

Serial.print(
  "NO BACKEND ACK FOR "
);
```

```
  Serial.println(
    recordID
  );

  return false;
}

// ============================================================
// WRITE TO DFR0602
// ============================================================

bool writeDFRRecord(
  String record
)
{
  dfrSerial.listen();

  while (
    dfrSerial.available()
  )
  {
    dfrSerial.read();
  }

  Serial.println();
  Serial.println(
    "DFR WRITE START"
  );
```

```
dfrSerial.print(
  "WRITE,"
);

dfrSerial.println(
  record
);

unsigned long start =
  millis();

while (
  millis() - start < 5000
)
{
  if (
    dfrSerial.available()
  )
  {
    String response =
      dfrSerial.readStringUntil(
        '\n'
      );

    response.trim();

    Serial.print(
```

```
  "DFR0602: "
);

Serial.println(
  response
);

if (
  response == "WRITE_OK"
)
{
  Serial.println(
    "SD WRITE SUCCESS"
  );

  return true;
}

if (
  response == "WRITE_FAIL"
)
{
  Serial.println(
    "SD WRITE FAILED"
  );

  return false;
}
```

```
    }

    yield();
  }

  Serial.println(
    "DFR WRITE TIMEOUT"
  );

  return false;
}

// ============================================================
// OFFLINE SYNC START
// ============================================================

bool startOfflineSync()
{
  if (
    WiFi.status() != WL_CONNECTED
  )
  {
    return false;
  }

  if (
    !mqttClient.connected()
  )
```

```
 {
  return false;
 }

 Serial.println();
 Serial.println(
  "========================================"
 );

 Serial.println(
  "OFFLINE SYNC STARTED"
 );

 Serial.println(
  "========================================"
 );

 return readDFRQueue();
}

// ============================================================
// READ OFFLINE QUEUE
// ============================================================

bool readDFRQueue()
{
 dfrSerial.listen();
```

```
while (
  dfrSerial.available()
)
{
  dfrSerial.read();
}

Serial.println(
  "SENDING READ TO DFR0602"
);

dfrSerial.println(
  "READ"
);

unsigned long globalStart =
  millis();

bool readStarted =
  false;

bool completed =
  false;

while (
  millis() - globalStart < 60000
)
{
```

```
// ---------------------------------------------------------
// Connection check
// ---------------------------------------------------------

if (
  WiFi.status() != WL_CONNECTED ||
  !mqttClient.connected()
)
{
  Serial.println(
    "CONNECTION LOST DURING SYNC"
  );

  return false;
}

mqttClient.loop();

// ---------------------------------------------------------
// DFR data
// ---------------------------------------------------------

if (
  dfrSerial.available()
)
{
  String line =
    dfrSerial.readStringUntil(
```

```
    '\n'
  );

line.trim();

if (
  line.length() == 0
)
{
  continue;
}

Serial.print(
  "DFR0602: "
);

Serial.println(
  line
);

// ========================================================
// READ START
// ========================================================

if (
  line == "READ_START"
)
{
```

```
  readStarted =
    true;

  Serial.println(
    "DFR QUEUE READING..."
  );

  continue;
}

// ====================================================
// READ END
// ====================================================

if (
  line == "READ_END"
)
{
  Serial.println(
    "DFR QUEUE READ COMPLETE"
  );

  completed =
    true;

  break;
}
```

```
// ======================================================
// RECORD
// ======================================================

if (
 readStarted &&
 line.startsWith("NODE_ID=")
)
{
 uint32_t recordID =
  extractRecordID(
   line
  );

 if (
  recordID == 0
 )
 {
  Serial.println(
   "INVALID RECORD ID"
  );

  dfrSerial.println(
   "NACK"
  );

  continue;
 }
```

```
Serial.println();
Serial.println(
  "----------------------------------------"
);

Serial.print(
  "OFFLINE RECORD RECEIVED: "
);

Serial.println(
  recordID
);

Serial.println(
  "----------------------------------------"
);

Serial.println(
  line
);

// ------------------------------------------------
// Publish to backend
// ------------------------------------------------

bool delivered =
  publishOnlineRecord(
```

```
    line,
    recordID
  );

if (
  delivered
)
{
  Serial.print(
    "OFFLINE RECORD DELIVERED: "
  );

  Serial.println(
    recordID
  );

  // Tell DFR:
  // this record is successfully processed.
  dfrSerial.println(
    "ACK"
  );

  Serial.println(
    "DFR ACK SENT"
  );

  delay(100);
}
```

```
  else
  {
    Serial.print(
      "OFFLINE RECORD FAILED: "
    );

    Serial.println(
      recordID
    );

    // Keep record in queue.
    dfrSerial.println(
      "NACK"
    );

    Serial.println(
      "DFR NACK SENT"
    );

    return false;
  }
}
else
{
  Serial.println(
    "UNKNOWN DFR DATA"
  );
```

```
    dfrSerial.println(
      "NACK"
    );

    return false;
   }
  }

  yield();
}

// ============================================================
// TIMEOUT
// ============================================================

if (!completed)
{
  Serial.println(
    "OFFLINE SYNC TIMEOUT"
  );

  return false;
}

// ============================================================
// ALL RECORDS ACKED
//
// Now clear DFR queue.
```

```
// ========================================================

delay(200);

while (
  dfrSerial.available()
)
{
  dfrSerial.read();
}

Serial.println();
Serial.println(
  "SENDING CLEAR TO DFR0602"
);

dfrSerial.println(
  "CLEAR"
);

unsigned long clearStart =
  millis();

while (
  millis() - clearStart < 5000
)
{
  if (
```

```
  dfrSerial.available()
)
{
  String response =
    dfrSerial.readStringUntil(
      '\n'
    );

  response.trim();

  Serial.print(
    "DFR0602: "
  );

  Serial.println(
    response
  );

  if (
    response == "CLEAR_OK"
  )
  {
    Serial.println();
    Serial.println(
      "======================================="
    );

    Serial.println(
```

```
      "OFFLINE QUEUE CLEARED"
    );

    Serial.println(
      "========================================"
    );

    return true;
   }

   if (
    response == "CLEAR_FAIL"
   )
   {
    Serial.println(
      "DFR CLEAR FAILED"
    );

    return false;
   }
 }

 yield();
}

Serial.println(
  "DFR CLEAR TIMEOUT"
);
```

```
 return false;
}

// ============================================================
// EXTRACT RECORD ID
// ============================================================

uint32_t extractRecordID(
 String record
)
{
 int start =
  record.indexOf(
   "RECORD_ID="
  );

 if (
  start < 0
 )
 {
  return 0;
 }

 start += 10;

 int end =
  record.indexOf(
```

```
    '',
    start
  );

 String idText;

 if (
  end < 0
 )
 {
  idText =
    record.substring(
      start
    );
 }
 else
 {
  idText =
    record.substring(
      start,
      end
    );
 }

 idText.trim();

 return idText.toInt();
}
```

```
// ============================================================
// CREATE RECORD
// ============================================================

String createRecord(
  uint32_t recordID
)
{
  String record = "";

  record +=
    "NODE_ID=";

  record +=
    NODE_ID;

  record +=
    ",RECORD_ID=";

  record +=
    String(recordID);

  record +=
    ",UPTIME_MS=";

  record +=
    String(millis());
```

```
record +=
 ",GPS_FIX=";

record +=
 gpsFix ? "1" : "0";

record +=
 ",LAT=";

record +=
 String(
  gpsLat,
  6
 );

record +=
 ",LON=";

record +=
 String(
  gpsLon,
  6
 );

record +=
 ",SAT=";
```

```
record +=
 String(
  gpsSat
 );

record +=
 ",TEMP=";

record +=
 String(
  dhtTemp,
  2
 );

record +=
 ",HUM=";

record +=
 String(
  dhtHum,
  2
 );

record +=
 ",PRESSURE=";

record +=
 String(
```

```
  pressure,
  2
 );

record +=
 ",BMP_TEMP=";

record +=
 String(
  bmpTemp,
  2
 );

record +=
 ",ACC_X=";

record +=
 String(
  accX,
  2
 );

record +=
 ",ACC_Y=";

record +=
 String(
  accY,
```

```
  2
 );

record +=
 ",ACC_Z=";

record +=
 String(
  accZ,
  2
 );

record +=
 ",GYRO_X=";

record +=
 String(
  gyroX,
  2
 );

record +=
 ",GYRO_Y=";

record +=
 String(
  gyroY,
  2
```

```
  );

 record +=
  ",GYRO_Z=";

 record +=
  String(
   gyroZ,
   2
  );

 record +=
  ",TAMPER=";

 record +=
  tamperState ? "1" : "0";

 return record;
}

// ============================================================
// DISPLAY SERVER
// ============================================================

void handleDisplayServer()
{
 WiFiClient client =
  displayServer.available();
```

```
if (!client)
{
  return;
}

unsigned long timeout =
  millis();

while (
  !client.available() &&
  millis() - timeout < 1000
)
{
  yield();
}

if (
  !client.available()
)
{
  client.stop();
  return;
}

String request =
  client.readStringUntil(
    '\r'
```

```
  );

client.flush();

// ============================================================
// DISPLAY DATA
// ============================================================

String response = "";

response +=
  "NODE_ID=";

response +=
  NODE_ID;

response +=
  ",RECORD_ID=";

response +=
  String(
    nextRecordID > 0
    ? nextRecordID - 1
    : 0
  );

response +=
  ",TEMP=";
```

```
response +=
 String(
  dhtTemp,
  1
 );

response +=
 ",HUM=";

response +=
 String(
  dhtHum,
  1
 );

response +=
 ",PRESSURE=";

response +=
 String(
  pressure,
  1
 );

response +=
 ",BMP_TEMP=";
```

```
response +=
 String(
  bmpTemp,
  1
 );

response +=
 ",GPS_FIX=";

response +=
 gpsFix ? "1" : "0";

response +=
 ",LAT=";

response +=
 String(
  gpsLat,
  6
 );

response +=
 ",LON=";

response +=
 String(
  gpsLon,
  6
```

```
  );

response +=
  ",SAT=";

response +=
  String(
    gpsSat
  );

response +=
  ",TAMPER=";

response +=
  tamperState ? "1" : "0";

response +=
  ",MQTT=";

response +=
  mqttClient.connected()
  ? "1"
  : "0";

// ============================================================
// HTTP RESPONSE
// ============================================================
```

```
  client.println(
   "HTTP/1.1 200 OK"
  );

  client.println(
   "Content-Type: text/plain"
  );

  client.println(
   "Connection: close"
  );

  client.println();

  client.println(
   response
  );

  delay(1);

  client.stop();
 }
```